Assignment 2:

Hugging Face Fine-Tuning, Experiment Tracking & Model Deployment

MLOps | PGD AI Programme | IIT Jodhpur

Total Marks: 80

Objective:

Build a complete MLOps workflow: take a Jupyter notebook, convert it to production scripts, track experiments with Weights & Biases, and publish your model to Hugging Face Hub.

Prerequisites:

- Python (intermediate level)
- Basic Machine Learning concepts (training, evaluation, metrics)
- Git and GitHub
- Hugging Face account (free at huggingface.co)
- Weights & Biases account (free at wandb.ai)

Assignment Tasks:

Task 1: Download the Starter Notebook

Download the notebook from the link below and place it in your project working directory.

Notebook link:
https://colab.research.google.com/drive/15yJsCtRu4kgqCLT44Tjhs3SFOT5GITqC?usp=sharing

Before anything else, run the notebook end-to-end in Colab to make sure it works and you understand each section at a high level.

Task 2: Convert Notebook to Python Scripts [10 marks]

Production ML systems do not run from Jupyter notebooks. Convert the notebook into clean, modular Python scripts.

- Create the following four script files [6 marks]
  - data.py — data loading, sampling, train/test split, encoding
  - train.py — model loading, Trainer setup, training loop
  - eval.py — evaluation, metrics, saving results

    - utils.py — shared helpers (label maps, dataset class, compute metrics)
- Remove all notebook-specific code (magic commands, display outputs) [2 marks]
- Scripts run without errors from the command line [2 marks]

 Tip: Use jupyter nbconvert --to script notebook.ipynb as a starting point, then clean up and split the output into the four modules above.

Task 3: Load a Pre-trained Model from Hugging Face [10 marks]

Hugging Face Hub hosts thousands of pre-trained models. You will load one and document your choice. **The Pre-Trained Model can be the same as per the given. ipynb file.**

- Load the tokenizer for your chosen model [3 marks]
- Load the model with the correct number of output labels [3 marks]
- Write a short paragraph (100–150 words) in the report explaining why you chose this model [4 marks]

Example: Loading DistilBERT

```
from transformers import DistilBertTokenizerFast,
DistilBertForSequenceClassification

model_name = 'distilbert-base-cased'
max_length = 512

tokenizer = DistilBertTokenizerFast.from_pretrained(model_name)
model = DistilBertForSequenceClassification.from_pretrained(
   model_name,
    num_labels=len(id2label)   # number of genre classes
).to(device)
```

 Tip: DistilBERT is 40% smaller and 60% faster than full BERT with only a small accuracy drop. You can use this model for this assignment.

Task 4: Train the Model & Track with W&B [25 marks]

This is the core task. You will fine-tune your model using the Hugging Face Trainer API and log everything to W&B.

- Prepare and encode the dataset correctly [5 marks]
- Configure Training Arguments including report_to="wandb" [5 marks]
- Define compute_metrics to return accuracy and F1 [5 marks]
- Run training successfully and confirm W&B run appears in your dashboard [5 marks]
- Include a screenshot of your W&B training dashboard in the report [5 marks]

Complete Training Script with W&B for Reference

```
import wandb
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, f1_score
```

```
# 1. Initialise W&B — log all your hyperparameters here
wandb.init(
    project="mlops-assignment2",
    name="distilbert-run-1",
   config={
          "model": model_name,
        "epochs": 3,
        "batch_size": 16,
        "learning_rate": 3e-5,
        "max_length": max_length,
        "dataset": "UCSD Goodreads",
   }
)

# 2. Define evaluation metrics
def compute_metrics(pred):
   labels = pred.label_ids
   preds  = pred.predictions.argmax(-1)
   return {
        "accuracy": accuracy_score(labels, preds),
        "f1":       f1_score(labels, preds, average="weighted")
   }

# 3. Training arguments — report_to="wandb" does the logging automatically
training_args = TrainingArguments(
    output_dir="./results",
   num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
   warmup_steps=100,
   weight_decay=0.01,
   logging_steps=50,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    report_to="wandb",       # <-- single line enables full W&B logging
    run_name="distilbert-run-1",
)

# 4. Train
trainer = Trainer(
   model=model,
   args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
trainer.train()

# 5. Always end the run
wandb.finish()
```

W&B logs the following automatically when report_to="wandb" is set:

What W&B Captures	Why It Matters
Training loss (every 50 steps)	Tells you if the model is learning

Validation loss (each epoch)	Tells you if the model is overfitting
Accuracy & F1 (each epoch)	Your main evaluation metrics
Learning rate schedule	Shows how LR changes over training
GPU/CPU utilisation	Helps diagnose performance bottlenecks
All hyperparameters	Makes every run fully reproducible

Task 5: Evaluate & Save Results [15 marks]

Run a final evaluation on the test set, record your metrics, and save them as a W&B Artifact.

- Run evaluation on the test set [3 marks]
- Record Accuracy, F1 score, and Loss [4 marks]
- Log final metrics to W&B explicitly [4 marks]
- Save the classification report as a file and upload as a W&B Artifact [4 marks]

Evaluation Script for Reference

```
import json, wandb
from sklearn.metrics import classification_report

# Run evaluation
eval_results = trainer.evaluate()
print(eval_results)

# Log final metrics to W&B
wandb.log({
    "final/loss":     eval_results["eval_loss"],
    "final/accuracy": eval_results["eval_accuracy"],
    "final/f1":       eval_results["eval_f1"],
})

# Save full classification report
preds  = trainer.predict(test_dataset).predictions.argmax(-1)
labels = [item["labels"].item() for item in test_dataset]
report = classification_report(labels, preds, target_names=list(id2label.values()),
output_dict=True)

with open("eval_report.json", "w") as f:
   json.dump(report, f, indent=2)

# Upload to W&B as a versioned Artifact
artifact = wandb.Artifact("eval-report", type="evaluation")
artifact.add_file("eval_report.json")
wandb.log_artifact(artifact)
wandb.finish()
```

Task 6: Save Model to Hugging Face Hub [10 marks]

Push your trained model to your public Hugging Face profile so it can be loaded by anyone, anywhere.

- Create or log in to your Hugging Face account [0 marks] — prerequisite
- Push the trained model weights to your HF profile [3 marks]
- Push the tokenizer to the same repository [3 marks]
- Ensure the repository is publicly accessible [2 marks]
- Log the HF model URL into your W&B run summary [2 marks]

Pushing to Hugging Face for Reference

```
from huggingface_hub import login

login(token="your_hf_token")   # or set HF_TOKEN env variable

# Push model and tokenizer
model.push_to_hub("your-username/distilbert-goodreads-genres")
tokenizer.push_to_hub("your-username/distilbert-goodreads-genres")

# Record the link in W&B
wandb.run.summary["huggingface_model"] = \
    "https://huggingface.co/your-username/distilbert-goodreads-genres"
```

 Tip: Store your Hugging Face token securely as an environment variable: export HF_TOKEN=your_token

Task 7: Push to GitHub [10 marks]

Your GitHub repository is the single source of truth for the entire project. It should let anyone reproduce your results from scratch.

- Push all Python scripts and requirements.txt [4 marks]
- Write a clear README.md [4 marks]
- Include evaluation results table in the README [2 marks]

README must include:

- Project description (one paragraph)
- Setup instructions (how to install dependencies and run the scripts)
- Results table with Accuracy, F1, and Loss
- Link to your Hugging Face model
- Link to your W&B project dashboard

Example Results Table for README

```
## Results

| Metric   | Score   |
|-----------|--------|
| Accuracy  | 0.XX   |
| F1 Score  | 0.XX   |
| Eval Loss | 0.XX   |

- Hugging Face model: https://huggingface.co/your-username/...
- W&B dashboard:      https://wandb.ai/your-username/mlops-assignment2
```

Submission Requirements:

- GitHub Repository Link — must be public
- Hugging Face Model Link — must be publicly accessible
- W&B Project Dashboard Link — set project visibility to public in W&B settings
- All all the links should be added in the report
- Plagiarism is strictly prohibited. Any student found will get zero marks
- A Final Report (PDF) —4-5 pages covering the four points below

The report should cover:

- Model selection rationale — why did you choose this model?
- Training summary — describe the training process and what the W&B charts showed
- Evaluation results — interpret your Accuracy, F1, and Loss numbers
- Challenges & learnings — what was hard? What would you do differently?

A Note to Students:

The Target for this assignment is not to achieve perfect accuracy but rather to understand the workflow of MLOps. Also, this assignment uses a BERT-based language model. You are not expected to understand how BERT works internally — that belongs to a Deep Learning course. Think of it as a powerful, pre-trained text engine that someone else has already built.

Your job in this MLOps assignment is to:

- Operationalize that model — track it with W&B and publish it
- Treat the model as a black box with inputs (text) and outputs (genre predictions)
- Focus on the workflow around the model, not the model's internals

GPU note: Use Google Colab (free tier) for training — it provides a GPU (recommended). If running on CPU, change `device_name = 'cpu'` and reduce sample size to 200 reviews per genre to keep training time manageable.